

ECS Project

Complete Release Readiness and Implementation Report

Code review, architecture assessment, remediation plan, release gates, and certification strategy

Current classification: RESEARCH ONLY. Improved runtime security, but not yet paper-certified and not live-trade ready.

Document scope	Repository-wide static review and implementation roadmap
Code reviewed	Approximately 11,900 lines of Python plus tests, configurations, and design documents
Prepared	2026-09-01
Workspace	/home/shrinivas/ECS_Project

Purpose

This report consolidates the complete technical assessment and provides an actionable way forward. Each recommendation is expressed as a release gate with implementation steps, required tests, and objective pass criteria.

1. Executive summary

The repository contains substantial research, calibration, control-system, and risk-management work. However, the executable paths are incomplete and several safety controls are either isolated, duplicated, or not enforced by the actual broker-submission boundary.

The primary issue is not one isolated defect. The project remains below release threshold because operating-mode safety, authorization integrity, transactional execution, model validity, portfolio accounting, build reproducibility, and paper certification have not all passed together.

Current classification

Capability	Assessment
Research and offline analysis	Usable with cleanup and reproducibility improvements
Deterministic simulation	Candidate; requires durable state, fake-broker scenarios, and recovery tests
Paper trading	Not certification-ready
Live trading	Not ready; must remain disabled

Positive foundations

- Clear awareness of research-versus-production boundaries in several scripts and reports.
- Authorization concepts include expiry, snapshots, HMAC signing, and single-use intent.
- Execution design recognizes partial fills, unknown submission state, reconciliation, and pre/post-sync checks.
- Risk and concentration concepts are separated sufficiently to support consolidation.
- Chronological model splits are directionally appropriate for time-series work.

Primary release blockers

- No single authoritative operating-mode and live-startup policy.
- Authorization is not guaranteed to be bound to the exact submitted order at the final gateway.
- Execution truth is not fully durable, transactional, idempotent, and restart-safe.
- Portfolio cash and exposure are not yet ledger-derived and atomically reserved.
- The model pipeline has target-definition and leakage risks; current economic validation is not a true strategy-return test.
- Repository layout, imports, dependencies, CI, and documentation are not reproducible as one deployable application.
- Formal deterministic simulation, crash recovery, fault injection, and paper soak certification remain outstanding.

2. Consolidated code, implementation, and design findings

Critical execution and authorization risks

- The unified execution gateway performs a reduced local token check instead of delegating to one authoritative authorization service.
- A token is not conclusively matched against every submitted order field: account, intent, symbol, side, quantity, order type, and price.
- Token signing and token verification must use exactly the same canonical payload; any mismatch makes cryptographic validation unreliable.
- Missing signing material must fail closed outside isolated tests.
- In-memory SQLite cannot provide restart recovery, durable audit, or reliable idempotency.
- An order may reach the broker before local journal persistence completes. Returning 'rejected' after such a failure creates duplicate-order risk on retry.
- Fill quantity and average fill price must come from broker trade records, never requested quantity or a placeholder price.
- Post-trade reconciliation must fail closed and quarantine the affected symbol/account when authoritative state is unavailable.

Model and research validity risks

- The target is selected by numeric column position instead of a named, versioned data contract.
- The target can remain in the model feature list, causing direct leakage and invalid validation metrics.
- Prediction-mean error is not an economic return measure and must not be used as a strategy-release criterion.
- Estimators are saved without complete preprocessing state, feature schema, data fingerprint, code version, and artifact checksum.

Portfolio and risk risks

- Minimum-quantity logic must not convert zero safe capacity into one share.
- Cash must be derived from a durable ledger and active reservations, not a mutable cached field.
- Risk and concentration checks must be part of the same atomic mutation that creates a reservation.
- Currency, fees, cash, notional, and P&L should use Decimal with a declared rounding policy.

Build, testing, and design risks

- Several runtime and test imports refer to missing modules.
- The README documents files and commands not present in the repository.
- The flat repository combines source, dated reports, generated results, research artifacts, and multiple competing implementation variants.
- There is no locked dependency manifest or clean-environment CI path.
- Broad exception handling often converts faults into empty results or apparent continuation, increasing false-success risk.
- The dashboard is appropriately labelled research-only, but duplicate symbols, bare exceptions, and ambiguous refresh behavior reduce operational reliability.

3. The 12-point priority roadmap

#	Priority	Required outcome
1	Non-live safety boundary	Research, backtest, paper, and live modes are centrally enforced; live is disabled by default.
2	Authorization integrity	One canonical, signed, order-bound, account-bound, single-use token is atomically consumed.
3	Durable execution storage	Intent, submission, broker order, fill, state transition, and incident history survive restart.
4	Real fills and reconciliation	Broker fills are authoritative; uncertainty quarantines risk and triggers reconciliation.
5	Leakage-safe model pipeline	Named target, explicit features, purged time splits, sealed holdout, and realistic costs.
6	Versioned model artifacts	Preprocessing, model, schema, data/code provenance, and checksums ship together.
7	Atomic portfolio accounting	Ledger-derived cash/exposure, reservations, Decimal arithmetic, and concurrency safety.
8	Dependency completeness	Every authoritative module exists; obsolete references are removed or archived.
9	Reproducible CI	Locked dependencies, lint, typing, tests, security checks, and coverage gates.
10	Repository consolidation	One package tree and one authoritative implementation per subsystem.
11	Accurate documentation	Every documented command and file exists and is tested where practical.
12	Formal certification	Simulation, restart recovery, fault injection, paper soak, and operator sign-off pass.

4. Gate 1 - Operating-mode safety foundation

Objective

Create one authoritative runtime policy controlling the broker environment and the permissions available to every entry point.

What to build

- Create `src/ecs/runtime/operating_mode.py`, `runtime_config.py`, and `startup_gate.py`.
- Define RESEARCH, BACKTEST, PAPER, and LIVE modes. Default to RESEARCH if no mode is supplied.
- Define a BrokerAdapter protocol and distinct SimulatedBrokerAdapter, PaperBrokerAdapter, and KiteBrokerAdapter implementations.
- Construct the adapter only through the startup gate. Business logic must not accept an arbitrary broker instance from untrusted callers.

```
class OperatingMode(str, Enum):  
    RESEARCH = 'research'  
    BACKTEST = 'backtest'  
    PAPER = 'paper'  
    LIVE = 'live'
```

Live startup checks

- Mode and broker adapter are compatible.
- Live trading is explicitly enabled; no implicit truthy fallback is accepted.
- Broker account ID matches the configured and authorized account.
- Signing key, durable execution database, kill switch, and risk policy are available.
- No unresolved reconciliation incident or incomplete recovery remains.
- Paper-certification evidence exists and has not expired or been invalidated by a code/configuration change.

How this overcomes the problem

The system gains a single fail-closed boundary. Research and paper processes cannot accidentally construct or import a live submission path, and a live process cannot start without durable evidence that all safety dependencies are ready.

Proof required

- Research mode rejects the live adapter.
- Live mode fails without signing key, durable database, correct account, clean reconciliation, and certification record.
- Invalid or missing mode never selects LIVE.

Gate passes only when every executable entry point requires a valid startup certificate before order submission.

5. Gate 2 - Authorization and exact order binding

Objective

Make authorization cryptographically meaningful and impossible to reuse for a different order.

Canonical authorization payload

- Include token ID, account ID, intent ID, idempotency key, symbol, side, quantity, order type, limit price, snapshot version, issue/expiry times, and nonce.
- Serialize with sorted JSON keys and fixed separators. Sign and verify the identical byte sequence.

```
def canonical_bytes(payload):  
    return json.dumps(asdict(payload), sort_keys=True, separators=(',', ':')).encode('utf-8')  
  
def sign(payload, secret):  
    return hmac.new(secret, canonical_bytes(payload), hashlib.sha256).hexdigest()
```

Verification sequence immediately before submission

- Load the token from the authorization database.
- Verify authorization decision, expiry, signature, account, broker snapshot version, and every order field.
- Use `hmac.compare_digest` for constant-time signature comparison.
- Atomically mark the token consumed and reserve the execution intent.
- Reject zero, negative, malformed, replayed, missing, or unsigned tokens.

Important design rule

Remove duplicated private verification from the gateway. The gateway must call the authoritative authorization service. Risk reduction must be calculated by the risk engine rather than accepted as a caller-provided Boolean.

Proof required

- Mutating account, intent, symbol, side, quantity, order type, or price invalidates the token.
- Two concurrent consume attempts yield exactly one success.
- A token cannot be verified if its database record is absent.
- Signing-key absence blocks all non-test startup.

Gate passes when no broker adapter can receive an order without an exact, database-backed, signed, single-use authorization.

6. Gate 3 - Durable, transactional, restart-safe execution

Objective

Create one authoritative record of what the strategy intended, what was authorized, what was sent, what the broker accepted, what filled, and how the position was reconciled.

Required tables

- execution_intents, authorization_tokens, order_submissions, broker_orders, fills, position_ledger, cash_ledger, state_transitions, reconciliation_incidents, and startup_certificates.
- Apply unique constraints to intent_id, idempotency_key, broker_order_id, and broker_fill_id.

Required state machine

```
CREATED -> AUTHORIZED -> SUBMISSION_RESERVED -> SUBMITTED -> ACKNOWLEDGED  
-> PARTIALLY_FILLED -> FILLED  
Alternatives: REJECTED, CANCELLED, EXPIRED  
Recovery: SUBMISSION_UNKNOWN, RECONCILIATION_REQUIRED, QUARANTINED
```

- Declare every allowed transition explicitly and reject all unspecified transitions.
- Append an immutable state-transition record containing previous state, new state, reason, time, actor, and correlation ID.

Safe submission procedure

- Create a unique intent and idempotency key.
- Authorize the exact order.
- In one database transaction, consume the token and move the intent to SUBMISSION_RESERVED.
- Submit to the broker once.
- Persist the broker order ID and acknowledgement.
- If the response is lost, record SUBMISSION_UNKNOWN and reconcile before any retry.

Persistence recommendation

SQLite with WAL mode and FULL synchronous durability is acceptable for a single local execution process. Use PostgreSQL before introducing multiple writers or remote execution workers.

Proof required

- All state survives process and machine restart.
- Repeated idempotency keys cannot create duplicate submissions.
- An invalid transition is rejected and audited.
- Backup and restore reproduce the same active state.

Gate passes when terminating the process at any execution step cannot create an unexplained duplicate or irrecoverable disagreement.

7. Gate 4 - Broker-authoritative fills and reconciliation

Objective

Treat broker order and trade data as authoritative and ensure uncertainty stops new risk rather than guessing success.

Implementation

- Treat place_order as a submission acknowledgement only.
- Poll or stream actual order status and broker fills using the broker order ID.
- Calculate filled quantity from unique broker fill records and weighted average price from actual fill prices.

```
filled_qty = sum(fill.quantity for fill in fills)
avg_price = sum(fill.price * fill.quantity for fill in fills) / filled_qty if filled_qty else None
```

- Use broker_fill_id as a unique database key so repeated polling is idempotent.
- After every terminal or uncertain state, compare ledger-derived expected position against the broker position.
- On timeout, mark SUBMISSION_UNKNOWN. Search broker order history by ID, intent tag, account, symbol, side, quantity, and time window before retry.
- On mismatch or unavailable post-sync, create a reconciliation incident and quarantine the symbol or account.

Proof required

- Full, partial, rejected, cancelled, delayed, duplicate, and out-of-order broker responses are handled deterministically.
- Repeated fill polling never duplicates a fill.
- Network timeout after submission never causes an automatic duplicate order.
- Position mismatch blocks further risk until resolved.

Gate passes when every local fill corresponds to exactly one broker fill and every local position agrees with an authoritative broker snapshot.

8. Gate 5 - Leakage-safe model validation

Objective

Make the prediction target, features, time availability, validation splits, and economic decision criteria explicit and reproducible.

Versioned dataset contract

```
dataset_version: daily_fusion_v1
date_column: date
symbol_column: symbol
target:
name: fwd_return_1d
horizonBars: 1
features:
- momentum_20d
- volatility_20d
- volume_zscore
forbidden_features:
- fwd_return_1d
- future_close
```

- Never infer the target or feature list from numeric column order.
- Assert that the target and future-derived fields are absent from features.
- Sort by symbol and time before creating lags and forward returns.
- Confirm that every feature is available at the decision timestamp.

Validation design

- Use chronological training, purged validation, and sealed holdout partitions.
- Apply an embargo at least as long as the target horizon.
- Do not reuse the holdout for tuning, early stopping, feature selection, or repeated debugging.
- Add automated leakage checks for target inclusion, negative feature shifts, future-prefixed fields, and overlapping records.

Economic evaluation

- Convert predictions into explicit signals and positions.
- Simulate actual turnover, brokerage, taxes, fees, spread, slippage, and capacity constraints.
- Report net return, volatility, Sharpe, Sortino, drawdown, turnover, hit rate, profit factor, exposure, and performance by time period and symbol.

Proof required

- Moving CSV columns does not change model inputs.
- Inserting the target or a future field into features fails the pipeline.
- All validation and holdout dates respect purge and embargo boundaries.
- Economic approval is based on net strategy results after realistic costs.

Gate passes when an independent reviewer can reproduce a sealed holdout result without using holdout information during development.

9. Gate 6 - Versioned and reproducible model artifacts

Objective

Ship preprocessing, estimator, schema, provenance, and validation evidence as one indivisible inference artifact.

Implementation

- For Ridge, use an sklearn Pipeline containing imputation, scaling, and the estimator.
- For XGBoost, use its native model format and package the fitted preprocessing pipeline alongside it.
- Store model file, manifest, feature schema, validation report, and checksums together.

```
artifact_version: model_0_v1.0.0
dataset_version: daily_fusion_v1
target: fwd_return_1d
features: [...]
training_start: 2024-01-01
training_end: 2025-04-30
git_commit: ...
artifact_sha256: ...
```

- Record library versions, data fingerprint, configuration fingerprint, random seed, code commit, and training time.
- At inference, reject missing, extra, reordered, or incorrectly typed features.
- Verify artifact checksum before loading. Restrict pickle/joblib loading to signed, allow-listed local artifacts.

Proof required

- Fresh-process inference matches training-process inference.
- Missing or reordered features fail validation.
- Artifact tampering is detected.
- The exact training environment can be reconstructed from the manifest and lockfile.

Gate passes when one versioned artifact completely defines how raw validated inputs become predictions.

10. Gate 7 - Atomic portfolio, cash, and risk accounting

Objective

Make portfolio state a deterministic product of durable fills and ledger entries, with safe reservations protecting concurrent order flow.

Ledger design

- Use append-only entries for deposits, reservations, releases, settlements, fees, taxes, realized P&L, dividends, and audited adjustments.

```
available_cash = settled_cash - active_cash_reservations
```

- Remove independently editable cached cash. Derive cash and exposure from the ledger and active reservations.
- Use Decimal for price, notional, cash, fees, and P&L with a documented rounding policy.

Atomic reserve-and-open workflow

- Within one transaction, read versioned portfolio state and validate cash, symbol concentration, gross/net exposure, count limit, per-trade risk, daily risk, drawdown, and leverage.
- Calculate the most restrictive safe quantity. If it is below the minimum, return zero; never manufacture one share.
- Create cash and exposure reservations, then commit before order submission.
- Settle the filled portion, retain reservation for remaining active quantity, and release unused reservation after cancellation or rejection.
- Allow position changes only from reconciled fills, approved corporate actions, or audited operator adjustments.

Proof required

- Zero capital and zero risk capacity yield zero quantity.
- Concurrent orders cannot spend the same cash or exposure capacity.
- Partial fills, cancellation, rejection, and restart preserve correct cash and positions.
- No public method can bypass risk checks by directly inserting a position.

Gate passes when cash, exposure, and positions can be rebuilt exactly from durable ledger records after restart.

11. Gates 8-11 - Repository and delivery integrity

Gate 8 - Resolve missing and competing components

- Create a dependency inventory listing each module, its callers, presence, status, ownership, and resolution.
- Restore required modules, redirect renamed imports, and archive obsolete components.
- Resolve known references including OrderImbalanceCore, KiteOrderImbalanceConnector, ECS_TradingSupervisor_Enhanced, block_5_risk_manager, block_7a_execution_intent_journal, and README observatory files.
- Use package-qualified imports and add import smoke tests for all production modules.

Gate 9 - Reproducible dependency and CI environment

- Create pyproject.toml and a lockfile with core, broker, ML, dashboard, and development dependency groups.
- Run syntax/import checks, Ruff, type checking, pytest, integration tests, recovery tests, security checks, leakage tests, and coverage gates in CI.

```
uv sync --all-groups
uv run pytest
uv run ruff check .
uv run pyright
```

- Use a declared supported Python version and deterministic fixtures. Separate broker/network tests from unit tests.

Gate 10 - Consolidate repository architecture

```
src/ecs/
authorization/ broker/ config/ data/
execution/ models/ portfolio/
reconciliation/ runtime/ dashboard/
tests/ configs/ docs/ scripts/ artifacts/ var/
```

- Identify the authoritative implementation for each subsystem before moving files.
- Add characterization tests, then migrate one subsystem at a time.
- Move dated reports to docs/archive and generated files to artifacts or var.
- Importing a module must not train models, connect to services, or write files.

Gate 11 - Accurate documentation

- Replace the README with actual purpose, maturity, operating modes, installation, test, simulation, paper, dashboard, reconciliation, and shutdown commands.
- State explicitly that live trading remains prohibited until certification.
- Test documented commands in CI where practical.

Combined pass condition

A clean machine can clone, install, test, simulate, and operate the documented non-live system without missing files, implicit dependencies, or undocumented steps.

12. Gate 12 - Formal simulation and paper certification

Stage A - Deterministic simulated broker

- Support full fills, partial fills, rejection, cancellation, delay, lost acknowledgement, duplicate response, out-of-order fill, stale snapshot, and position mismatch.
- The same event script and seed must produce the same database, ledger, and final state.

Stage B - Crash and restart matrix

- Terminate after intent creation, authorization, token consumption, submission reservation, broker submission, partial fill, fill recording, and before reconciliation.
- Restart must recover without duplicate order, incorrect cash, incorrect position, invalid token state, or missing audit history.

Stage C - Fault injection

- Inject database outage/lock, disk full, Redis failure, broker timeout, stale market data, malformed payload, clock drift, signing-key failure, and reconciliation timeout.
- In every uncertain state, block new risk and preserve enough evidence for reconciliation.

Stage D - Sustained paper soak

- Run across multiple market sessions including open/close, volatility spikes, partial fills, reconnects, and illiquid symbols.
- Continuously compare intended, authorized, submitted, broker, filled, and reconciled orders; also compare broker/local positions and cash.

Required release evidence

Criterion	Required result
Authorization bypass	Zero
Unexplained duplicate orders	Zero
Unresolved position mismatches	Zero
Fill representation	Every broker fill appears exactly once locally
Restart recovery	All crash points recover deterministically
Kill switch	Demonstrated and audited
Daily-loss halt	Demonstrated and audited
Stale-data halt	Demonstrated and audited
Approvals	Model, reconciliation, risk, and operator sign-off

Gate passes only after the paper-soak evidence package is complete and independently reviewed.

13. Recommended implementation sequence

Phase	Scope	Deliverable
1 - Stop unsafe execution	Operating modes, startup gate, broker separation, live disabled	Fail-closed runtime boundary
2 - Establish execution truth	Authorization binding, durable store, state machine, idempotency, reconciliation	Restart-safe simulated execution core
3 - Establish portfolio truth	Ledgers, reservations, Decimal, partial-fill settlement	Atomic cash and exposure accounting
4 - Establish model truth	Dataset contract, leakage controls, purged validation, realistic costs, artifacts	Reproducible model evidence
5 - Establish build integrity	Package structure, missing imports, lockfile, CI, README	Clean reproducible application
6 - Certify	Simulation matrix, crash recovery, fault injection, paper soak	Paper-certification dossier

Suggested milestone classifications

- RESEARCH_ONLY - one or more engineering or model gates remain incomplete.
- SIMULATION_CERTIFIED - deterministic simulated execution and restart recovery pass.
- PAPER_CANDIDATE - engineering gates pass; live market paper soak is pending.
- PAPER_CERTIFIED - paper soak, reconciliation, and operating procedures pass.
- LIVE_CANDIDATE - independent risk/security review and live controls are pending.
- LIVE_CERTIFIED - every gate and operational approval passes.

Immediate target

Aim first for SIMULATION_CERTIFIED, then PAPER_CANDIDATE. Do not connect the live broker while execution truth, portfolio truth, model truth, and certification evidence remain incomplete.

14. Concrete work packages and definition of done

WP	Work package	Definition of done
WP-01	Runtime policy	All entry points require startup certificate; live disabled by default
WP-02	Broker abstraction	Simulation, paper, and live adapters pass contract tests
WP-03	Authorization v2	Canonical signing, order binding, atomic consumption, replay tests
WP-04	Execution store	Durable schema, strict state machine, unique idempotency constraints
WP-05	Fill/reconciliation service	Actual fills, unique fill IDs, quarantine, unknown-submission recovery
WP-06	Portfolio ledger	Decimal accounting, cash/exposure reservations, rebuild after restart
WP-07	Dataset contract	Named target/features, availability checks, leakage scanner
WP-08	Model evaluation	Purged splits, sealed holdout, realistic-cost strategy metrics
WP-09	Artifact registry	Model plus preprocessing, manifest, checksum, provenance
WP-10	Packaging	src layout, one authoritative module per subsystem, import-safe modules
WP-11	CI and environment	Lockfile, lint/type/test/security/recovery gates
WP-12	Documentation	All commands and file references verified
WP-13	Certification harness	Deterministic scenarios and crash/fault matrix automated
WP-14	Paper soak	Thresholds met with signed evidence and operator sign-off

Change-control rule

Any change to authorization, risk limits, execution state transitions, broker adapter, model features, target, preprocessing, or cost assumptions invalidates the relevant certification evidence and requires targeted recertification.

15. Final release checklist

Safety

- ☐ Operating mode is explicit and live is disabled by default.
- ☐ Startup gate passes and produces an auditable certificate.
- ☐ Kill switch, loss halt, stale-data halt, and reconciliation quarantine are proven.

Authorization and execution

- ☐ Every order matches an exact, signed, single-use authorization.
- ☐ Intent, token, submission, broker order, fill, and transition history are durable.
- ☐ Unknown submissions are reconciled before retry.
- ☐ Broker fills and positions agree with local ledgers.

Portfolio

- ☐ Cash and exposure are ledger-derived and reservations are atomic.
- ☐ Partial fills and cancellations settle and release correctly.
- ☐ All money uses Decimal and declared rounding rules.

Model

- ☐ Named target and explicit feature contract are enforced.
- ☐ Leakage, purge, embargo, holdout, and cost tests pass.
- ☐ Reproducible signed artifact contains preprocessing and provenance.

Delivery

- ☐ Clean checkout installs from the lockfile.
- ☐ All production imports, tests, lint, typing, security, and recovery checks pass.
- ☐ README commands and paths are valid.

Certification

- ☐ Deterministic simulation matrix passes.
- ☐ Crash/restart and fault-injection matrix passes.
- ☐ Paper-soak thresholds pass with zero unresolved critical incidents.
- ☐ Independent model, risk, reconciliation, and operator approvals are recorded.

Final conclusion: the correct near-term goal is a durable, deterministic, restart-safe simulated engine. Paper certification follows only after model and operational evidence pass. Live trading follows only after every gate is independently verified.

End of report